

Data Engineer Internship Task – Livesport

Data origin from one of our affiliate partners and contain information about users. For my disposition are two datasets players_data_v1.csv & players_data_v2.csv. In this document are excerpts of code, full code is with comments in .sql file. This document illustrates which method I have chosen for task and how I conclude task.

Step 1 – Load data in database

I will work in VS Code using DuckDB database. Codes are in attached .sql file.

```
CREATE TABLE players_v1 AS
```

```
SELECT * FROM read_csv_auto('players_data_v1.csv');
```

Step 2 – Find differences between tables

1. *Row count* – Is there difference in the row count between tables?

Method: For this I will use `SELECT COUNT(*) FROM players_v1`

Rows in v1 table	Rows in v2 table
197516	205070

Conclusion: In second table are 7554 more rows. The surplus may be caused by

- New/additional users
- Duplicate rows

Unfortunately without further analysis the true reason is unclear

2. *Missing values* – Are in provided columns some missing values?

Method: I will be firstly looking at **NULL values** using following code for each column (I also tried empty strings, where it makes sense based on data type).

```
COUNT(*) FILTER (WHERE as_of IS NULL) AS missing_as_of
```

	V1	V2
Tracker_id	0	12265
Tracker_name	982	11760
else	0	0

Conclusion: Problems arise at tracker_id and tracker_name. This suggest tracking or mapping inconsistency in both tables

Method: I am also interested, whether there are some other data that are in one table but are not in the other (where it makes sense – country, platform, signup_year, project_name. Other data – numerical, it is expectable they will differ and date is

already accounted in signup_year, I am interested in whether they are from same period).

```
SELECT DISTINCT country ..(platform...)
FROM players_v1
EXCEPT
SELECT DISTINCT country
FROM players_v2;
```

	Is in v1 and NOT in v2	Is in v2 and NOT in v1
country	New Zealand, Canada-Ontario	.
platform	Huawei	.
Signup_year	Years 2008-2026	9999
Project_name	.	.

Conclusion: In v2 are not included countries New Zealand and Canada-Ontario, This suggests incomplete country mapping or even lack of research from these countries. Also Canada-Ontario may be in Canada bracket. This suggests a minor bug in Country recognition.

Also in v1 dataset id Huawei as platform, in v2 Huawei doesn't appear.

Code discovered obvious problem with signup_year, since all of years in v2 are 9999.

I suggest fixing this bug using better separation of the year from signup_date.

3. *Duplicate records* – are in tables duplicate records?

Method: I am looking for 1:1 copies, since even if only one column is different than other, it provides new information, at worst there can be sorting or assigning problem, but we will be losing essential information.

```
SELECT COUNT(*) AS duplicate_groups
FROM (
    SELECT *
    FROM players_v1 -and v2
    GROUP BY ALL
    HAVING COUNT(*) > 1);
```

Duplicates	V1	V2
	39	8835

Conclusion: In first table are 39 duplicated row groups, in v2 are 8835 duplicated row groups. This may partly explain the reason why is the second dataset bigger than v1, however I would also like to verify whether there aren't some new users.

Additionally: I will find out how many rows are in v1 and not in v2, and in v2 and v1

```
SELECT COUNT(*) AS only_in_v2
FROM players_v2 v2
LEFT JOIN players_v1 v1
```

```

ON v2.user_id = v1.user_id
AND v2.as_of = v1.as_of
WHERE v1.user_id IS NULL;

```

	Only in v2	Only in v1
	0	1242

Conclusion: Every line with same as_of and user_id that is in v1 is also in v2, On the contrary, 1242 lines (with same user_id and as_of) are only in v1.

4. *Are numerical columns between tables distinct?*

Method: I will identify rows not only using user_id, since it can be expected people will be using app more often so more logins may be recorded. I have chosen time identifier as_of and user_id, since other information may be same in more columns and this would result in wrong identification of duplicate columns

```

SELECT
    COUNT(*) AS revenue_mismatches
FROM players_v1 v1
JOIN players_v2 v2
    ON v1.user_id = v2.user_id
    AND v1.as_of = v2.as_of
WHERE v1.revenue IS DISTINCT FROM v2.revenue;

```

First_deposit_amount	205610
Sports_bets_turnover	205610
Sportsbook_net_revenue	205610
Total_net_revenue	205610
Deposits	205610
Revenue	205609

Conclusion: Firstly, I notice that there is about 600 more rows in results, but that is caused by comparing rows to duplicates. Secondly, based on result I conclude that there is systematic shift in data in both tables. This may be caused by

- Different currency
- Different scaling

The mismatch count equals nearly all joined rows, indicating a systematic transformation rather than random data corruption.

5. *Are date and text columns consistent?*

I have already discovered that in v2 table there is an issue when transferring signup_year from signup_date, because there are only years 9999..

Table v1	Table v2
----------	----------

Method: This I will achieve looking into each

0

205070

database and extract sign up year from date and compare it to the signup_year column

```
SELECT COUNT(*) AS inconsistent_signup_year
```

```
FROM players_v2
```

```
WHERE signup_year != EXTRACT(YEAR FROM signup_date);
```

Conclusion: this result clearly states that all of signup_years in second table are not in agreement with year at signup_date. So perhaps there is wrong extraction from date or an tracking issue.

Addition: I have also checked, whether there aren't any transactions even before signup, which would incline an incorrect mapping or faulty transfer of information and also inconsistent first deposit year, both of these were okay

```
SELECT COUNT(*) AS inconsistent_first_deposit_year
```

```
FROM players_v2
```

```
WHERE first_deposit_year != EXTRACT(YEAR FROM  
first_deposit_date);
```

```
SELECT COUNT(*) AS invalid_deposit_before_signup
```

```
FROM players_v2
```

```
WHERE first_deposit_date < signup_date;
```

6. Are relations between columns correct?

Method: I will group data regarding tracker_id and in case one tracker_id has more than one tracker_name, it is inconsistency

```
SELECT COUNT(*) AS inconsistent_tracker_ids
```

```
FROM (
```

```
  SELECT tracker_id
```

```
  FROM players_v1 -- same for v2
```

```
  GROUP BY tracker_id
```

```
  HAVING COUNT(DISTINCT tracker_name) > 1);
```

	V1	V2
Tracker id	2	29
Tracker name	71	69

In table above: Tracker id means how many tracker ids have more than one tracker name. Tracker name means how many tracker names have more than one tracker id.

Conclusion: This analysis shows that the relationship between tracker_id and tracker_name is not consistent in either dataset. There are multiple cases where a single tracker_id is associated with more than one tracker_name, as well as cases where a single tracker_name is linked to multiple tracker_ids. This indicates a violation of the expected one-to-one mapping between these fields and suggests

issues in tracking logic or data mapping. Such inconsistencies can lead to incorrect attribution and should be investigated and resolved before further use of the data.

Additionally: I have also compared how many rows with same user_id, tracker_id and as_of have different tracker_name in both tables. 418 have different tracker name in v2 than in v1, this suggests tracking problem.

```
SELECT COUNT(*) AS tracker_mismatches
FROM players_v1 v1
JOIN players_v2 v2
ON v1.tracker_id = v2.tracker_id
AND v1.user_id = v2.user_id
AND v1.as_of = v2.as_of
WHERE v1.tracker_name IS DISTINCT FROM v2.tracker_name;
```

Key findings:

- Second table has 7554 more rows
- Both tables have duplicates (v1-39,v2-8835) >> large number of duplicates in v2
- In tables are NULL values at tracker_id (v1-0,v2-12265) and tracker_name (v1-982, v2- 12265) in both files >> possible tracking inconsistency
- The systematic differences suggest either different scaling or currency conversion >> v2 appears to be a transformation of v1
- In v1 table are more countries (Canada-Ontario, New Zealand), platforms (Huawei) >> possible mapping inconsistency
- In v2 is an issue with signup_year, only years 9999 >> invalid signup_year in v2
- In both tables exists trackers_id with more tracker_names and tracker_names with more tracker_ids >> possible tracking issue